Paralelizando algoritmos con CUDA

Sebastian Estrada - 21405
Adrián González - 23152
Fernando Mendoza - 19644

I. Introducción

El salto hacia la aceleración por GPU requiere un cambio de estrategia. A diferencia de la paralelización en memoria compartida, para paralelizar exitosamente en CUDA, el primer paso fundamental es examinar el programa secuencial e identificar sus secciones **SIMD** (*Single Instruction, Multiple Data*): aquellas áreas donde una misma operación se aplica simultáneamente a múltiples datos.

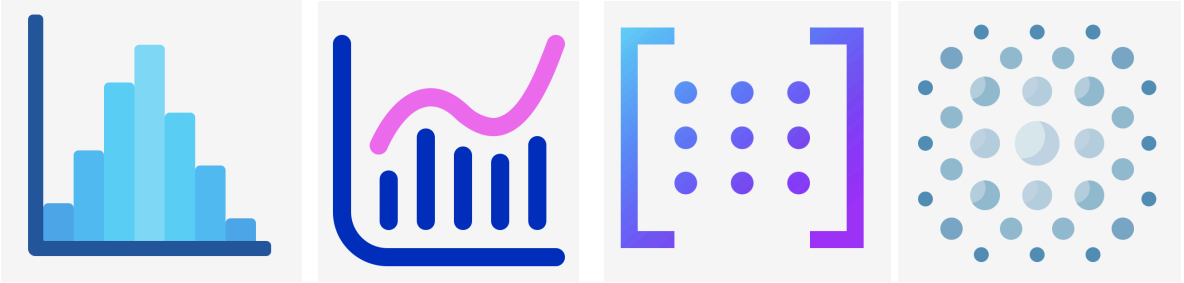
Para esta práctica contaremos con un conjunto de **4 algoritmos** adaptados para un flujo de trabajo en Python. Deberán **elegir uno de estos algoritmos** para desarrollar y comparar su versión secuencial en CPU contra su versión paralela en GPU. El objetivo central del análisis será comprender las diferencias arquitectónicas, los cuellos de botella de memoria y las particularidades de paralelizar bajo el modelo SIMD.

Histograma Masivo: Leer un arreglo con miles de millones de mediciones y contarlas sumando +1 en una de las 100 cubetas de clasificación disponibles.

Integración Numérica (Suma de Riemann): Calcular el área bajo una curva sumando matemáticamente el área individual de 10^9 rectángulos minúsculos independientes.

Multiplicación de Matrices Densas ($C = A \times B$): Multiplicar dos matrices cuadradas gigantes (un millón por un millón de elementos) calculando el producto punto de filas y columnas.

Filtro de Desenfoque de Imagen (Blur): Promediar el valor de cada píxel de una imagen satelital de 8K con sus 8 vecinos inmediatos para calcular su nuevo color en la matriz resultante.



Pasos Generales de la actividad:

1. **Preparación del entorno de trabajo:** poder trabajar con Python y la librería de CUDA en el equipo que tiene GPU para poder realizar las instrucciones.
2. **Identificación del algoritmo secuencial en Python:** Crear una versión secuencial del algoritmo con la que se utiliza de referencia el tiempo para el benchmarking.
3. **Creación de la solución con CUDA:** identificar las partes paralelizables. Utilizar CUDA para crear esas secciones.

4. **Análisis:** medir y discutir si los resultados mejoraron.

II. Preparación del entorno de trabajo:

A. Usando su propio equipo:

Activar el entorno virtual de Python e instalar las librerías necesarias para compilar código directamente hacia la arquitectura de la GPU

Instalar python3 si no está aún disponible.

```
pip install numpy numba cupy
```

Crear env con conda: al instalar Python pueden usar conda para crear un env de desarrollo

```
conda create --name nombre_de_su_env python=3.10 -y
```

Para activar

```
conda activate cuda_env
```

Dentro del env, instalar las siguientes librerías:

```
conda install -c conda-forge cudatoolkit numba cupy numpy -y
```

y para probar que está funcionando

```
from numba import
cuda cuda.detect()
```

Las instrucciones anteriores les permiten utilizar CUDA con python en su entorno local, si no funciona pueden investigar alguna librería o env alternativo

¿Les funcionó el set up del env local o debieron utilizar alguna alternativa? ¿Cuál utilizaron?

No se usó conda. Se utilizó como alternativa un entorno virtual de Python (python3 -m venv) con pip, instalando numba, numba-cuda, cupy-cuda12x y numpy. Así se obtiene el toolkit de CUDA 12 como paquetes de pip, sin necesidad de cudatoolkit de conda. Se verificó con cuda.detect() en una GPU NVIDIA RTX 3060 Laptop.

```
from numba import cuda
cuda.detect()
```

✓ 1.4s

```
Found 1 CUDA devices
id 0    NVIDIA GeForce RTX 3060 Laptop GPU                          [SUPPORTED]
                      Compute Capability: 8.6
                           PCI Device ID: 0
                              PCI Bus ID: 1
                                    UUID: GPU-2e9c7e14-1aee-452f-bf47-1b33b94c78c5
                                Watchdog: Enabled
             FP32/FP64 Performance Ratio: 32
                           FP16 Support: Yes
                           BF16 Support: Yes
                            FP8 Support: No
Summary:
        1/1 devices are supported

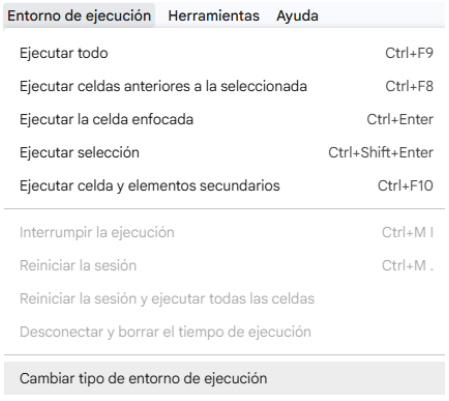
True
```

```
(cuda_env) PS C:\Users\adria\Documents\UVG\PPD\Prácticas> python -c "from numba import cuda; cuda.detect()"
Found 1 CUDA devices
id 0    NVIDIA GeForce RTX 3050 6GB Laptop GPU                      [SUPPORTED]
                      Compute Capability: 8.6
                           PCI Device ID: 0
                              PCI Bus ID: 1
                                    UUID: GPU-60780ab2-0ed7-be5b-0c06-23d1f9f65194
                                Watchdog: Enabled
                            Compute Mode: WDDM
             FP32/FP64 Performance Ratio: 64
                           FP16 Support: Yes
                           BF16 Support: Yes
                            FP8 Support: No
Summary:
        1/1 devices are supported
```

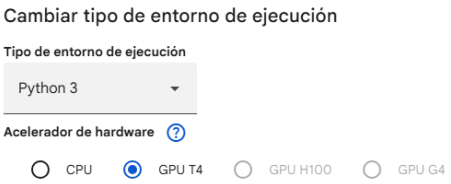
B. Utilizando google colab:

Dirigirse al siguiente enlace https://colab.research.google.com/ y asegurarse de haber iniciado sesión con una cuenta de Google. Crear un nuevo cuaderno de trabajo.

En la barra de menú superior, seleccionar la pestaña *Entorno de ejecución* y, posteriormente, hacer click en la opción *Cambiar tipo de entorno de ejecución:*

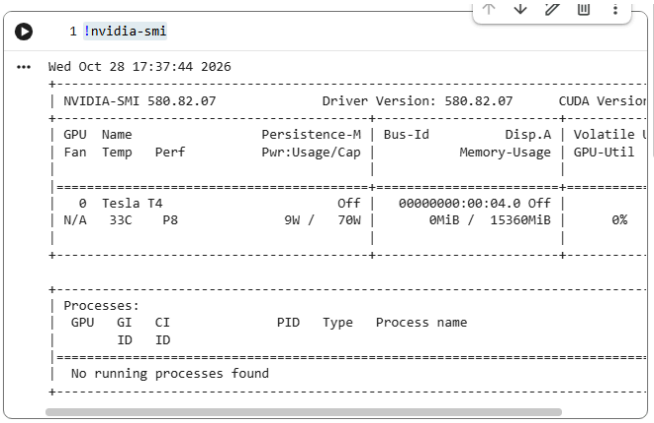


En la ventana de configuración, localizar la sección Acelerador de hardware, seleccionar la opción *GPU T4* y hacer clic en Guardar para aplicar los cambios.



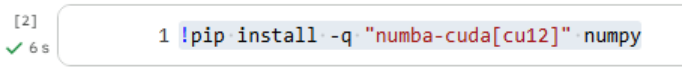
Una vez guardados los cambios, Google Colab estará configurado para utilizar una GPU T4, sujeta a disponibilidad.

A continuación, se verificará el acceso a la GPU y se prepararán las bibliotecas necesarias para desarrollar y ejecutar programas de procesamiento paralelo utilizando CUDA con Python.
Agregar una nueva celda de código en Google Colab e ingresar el siguiente comando: `!nvidia-smi`.
Ejecutar la celda para consultar la información de la GPU asignada al entorno de ejecución. El resultado mostrará datos como el modelo de la tarjeta gráfica, la versión del controlador NVIDIA, la versión de CUDA compatible con el controlador y la memoria disponible.



Crear una nueva celda de código e ingresar el siguiente comando: `!pip install -q "numba-cuda[cu12]" numpy`.
Ejecutar la celda para instalar las bibliotecas necesarias para desarrollar programas CUDA con Python.

- numba-cuda: permite desarrollar y ejecutar kernels CUDA mediante Python.
- numpy: permite crear y manipular arreglos numéricos que posteriormente podrán procesarse en la GPU.
- -q: reduce la cantidad de mensajes mostrados durante la instalación.
- Verificar que la instalación finalice sin errores.



A partir de este punto, pueden crear nuevas celdas de código para implementar y ejecutar las soluciones de los ejercicios propuestos en el ejercicio.

III. Identificación del algoritmo secuencial en Python

Cada grupo deberá elegir un algoritmo de la lista de problemas originales.

Para el algoritmo seleccionado, deberán analizar cómo se resolvería:

a. **Elaborar un diagrama de flujo para el algoritmo secuencial**, en el que se identifiquen claramente los datos de entrada, las operaciones necesarias, las decisiones y los resultados esperados.
b. **Completar la siguiente tabla:**

Nombre del algoritmo elegido	Multiplicación de Matrices densas
Justificación de la selección del algoritmo	Por que cada elemento C[i, j] se calcula de forma independiente aplicando la misma operación (un producto punto) sobre datos distintos, así que se asigna de forma natural a una Grid 2D de hilos
Identificación de los datos de entrada	Matrices, tamaño, tipo de dato (float), contenido
Identificación de los datos de salida	Matriz, Tamaño, tipo de dato y contenido de la matriz
Descripción de las operaciones y procesos involucrados	El algoritmo calcula C = A × B. Cada elemento C[i, j] es el producto punto de la fila i de A con la columna j de B. En CPU se calcula de forma secuencial con tres ciclos anidados. En GPU, A y B se copian a la memoria del dispositivo y cada hilo de una Grid 2D calcula un elemento de C leyendo los datos desde memoria global. Al final, C se copia de vuelta a la CPU y se compara con el resultado secuencial.

Codigo:

```python
import math
import time
import numpy as np
from numba import cuda

def multiplicar_secuencial(A, B):
    n = A.shape[0]
    C = np.zeros((n, n), dtype=np.float32)
    for i in range(n):
        for j in range(n):
            suma = np.float32(0.0)
            for k in range(n):
                suma += A[i, k] * B[k, j]
            C[i, j] = suma
    return C

@cuda.jit
def multiplicar_paralelo(A, B, C):
    i, j = cuda.grid(2)
```

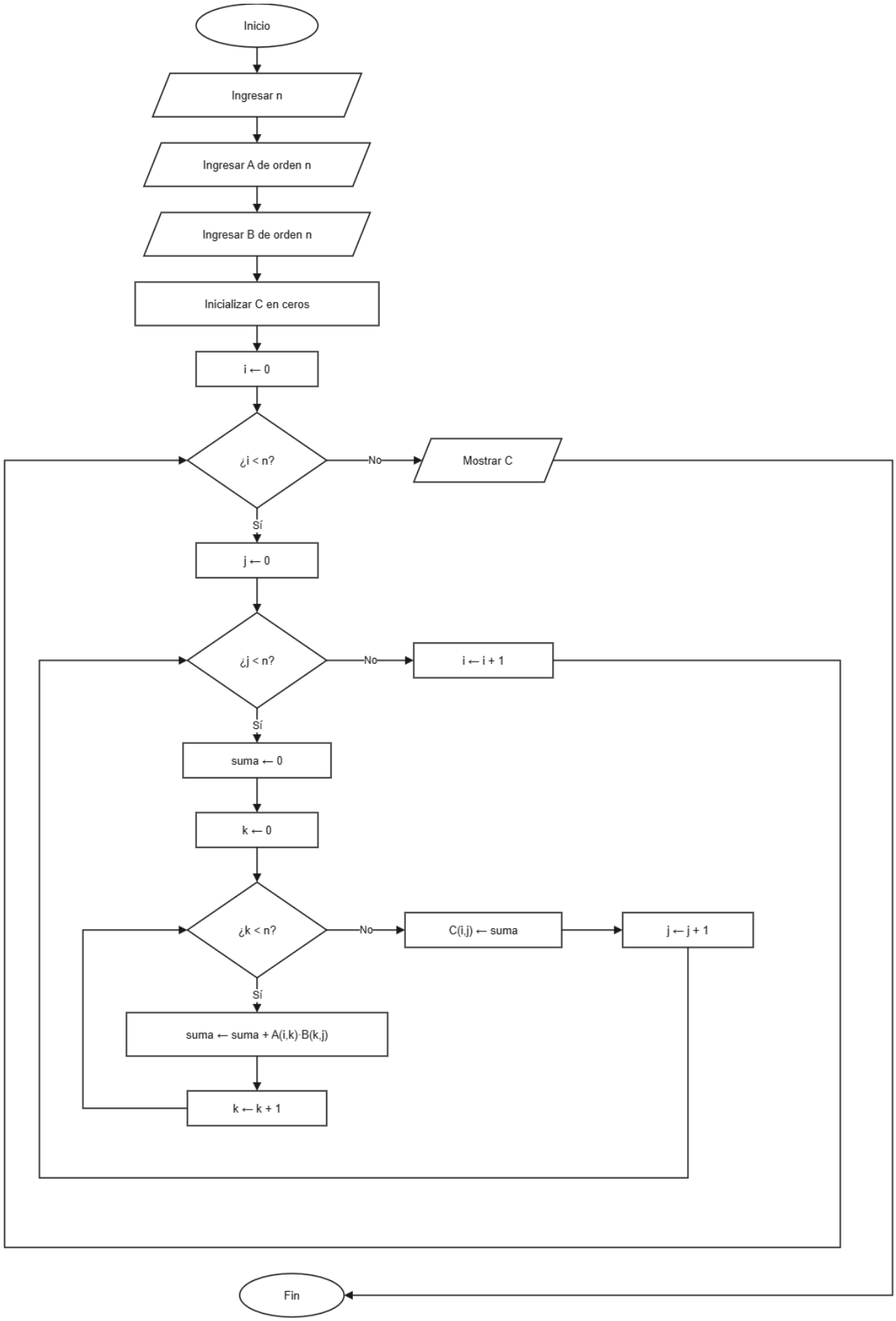
```
    n = A.shape[0]
    if i < n and j < n:
        suma = np.float32(0.0)
        for k in range(n):
            suma += A[i, k] * B[k, j]
        C[i, j] = suma

def main():
    cuda.detect()
    n = 256
    rng = np.random.default_rng(1)
    A = rng.random((n, n), dtype=np.float32)
    B = rng.random((n, n), dtype=np.float32)
    t0 = time.perf_counter()
    C_cpu = multiplicar_secuencial(A, B)
    t_cpu = time.perf_counter() - t0
    dA = cuda.to_device(A)
    dB = cuda.to_device(B)
    dC = cuda.device_array((n, n), dtype=np.float32)
    hilos = (16, 16)
    bloques = (math.ceil(n / 16), math.ceil(n / 16))

# La primera llamada compila el kernel.
    multiplicar_paralelo[bloques, hilos](dA, dB, dC)
    cuda.synchronize()
    t0 = time.perf_counter()
    multiplicar_paralelo[bloques, hilos](dA, dB, dC)
    cuda.synchronize()
    t_gpu = time.perf_counter() - t0
    C_gpu = dC.copy_to_host()

    print(f"secuencial: {t_cpu:.3f} s")
    print(f"paralelo:   {t_gpu:.4f} s")
    print(f"coincide:   {np.allclose(C_cpu, C_gpu, rtol=1e-3, atol=1e-3)}")
```

Diagrama de Flujo programa completo:


Inicio
Ingresar n
Ingresar A de orden n
Ingresar B de orden n
Inicializar C en ceros
i ← 0
¿i < n?
No
Mostrar C
Sí
j ← 0
¿j < n?
No
i ← i + 1
Sí
suma ← 0
k ← 0
¿k < n?
No
C(i,j) ← suma
j ← j + 1
Sí
suma ← suma + A(i,k)·B(k,j)
k ← k + 1
Fin

c. **Desarrollar la versión secuencial del algoritmo seleccionado utilizando Python, mediante arreglos básicos o NumPy.** No se permite utilizar funciones de bibliotecas que realicen procesamiento paralelo o que implementen directamente una versión optimizada del algoritmo.

Fuente: Numba Developers. Matrix multiplication. *Numba documentation, CUDA examples.* https://numba.readthedocs.io/en/stable/cuda/examples.html

Si utilizan como referencia una implementación existente, deberán citar la fuente correspondiente.

Indicar el nombre del archivo de Python que contiene la implementación correspondiente	sec_main.py

d. **Seleccionar o generar un conjunto de datos suficientemente grande para evaluar posteriormente el rendimiento de las versiones secuencial y paralela.**

Cantidad de datos que procesará el algoritmo	2 matrices de entrada (A y B) con 1000 × 1000 = 1 000 000 de elementos float32 cada una, o sea 2 000 000 de valores de entrada. El resultado es una matriz C de 1 000 000 de elementos. Cada elemento de C es un producto punto de 1000 términos, así que en total se hacen 1000^3 = 1 000 000 000 de multiplicaciones-sumas.
Dimensiones del conjunto de datos (cuando corresponda)	A: 1000 × 1000, B: 1000 × 1000, C: 1000 × 1000 (matrices cuadradas densas)
Tamaño total de los datos en memoria (bytes, KB o MB)	Cada float32 ocupa 4 bytes, así que cada matriz ocupa 1 000 000 × 4 = 4 000 000 bytes ≈ 3.81 MB. Las entradas A + B suman 8 000 000 bytes ≈ 7.63 MB. Contando también C, el total es 12 000 000 bytes ≈ 11.44 MB.

IV. Creación de la solución paralela usando CUDA

La primera diferencia al trabajar con una GPU es que el Host (CPU) y el Device (GPU) utilizan espacios de memoria separados. Transferir grandes cantidades de datos desde la memoria RAM del Host hacia la memoria de la GPU (VRAM) requiere tiempo. Si el cálculo es relativamente sencillo, el tiempo de transferencia podría superar el tiempo de procesamiento en la GPU, reduciendo o incluso eliminando los beneficios de la paralelización.

¿Cómo están organizados y representados los datos de entrada de su algoritmo? (¿De qué forma se ven sus datos de entrada?)

Los datos de entrada son dos matrices cuadradas densas, A y B, de 1000 × 1000 elementos de tipo float32 (números reales de precisión simple, 4 bytes cada uno). Se representan como arreglos bidimensionales de NumPy (np.ndarray) guardados de forma contigua en memoria, fila por fila (orden C), y sus valores son aleatorios uniformes entre 0 y 1. Cada matriz ocupa 4 000 000 bytes (≈ 3.81 MB), en total 8 000 000 bytes (≈ 7.63 MB). Para usarlas en la GPU se copian a la VRAM con cuda.to_device(), donde mantienen la misma forma bidimensional.

Utilizar los datos definidos para su algoritmo y las funciones correspondientes de CUDA Python. Medir el tiempo que tarda la transferencia utilizando un cronómetro en Python.

¿Cuánto tiempo tomó transferir los datos desde el Host hacia la GPU?

≈ 1.7 ms (promedio de 3 ejecuciones: 1.915, 1.715 y 1.571 ms)

En OpenMP, mediante la directiva `#pragma omp parallel for`, el entorno de ejecución distribuye las iteraciones entre los hilos. En CUDA, el programador debe definir la configuración de bloques e hilos de acuerdo con las características del problema.

Los hilos (*threads*) se agrupan en bloques (Blocks), con un máximo habitual de 1024 hilos por bloque, y los bloques forman una Grid.

Si el problema utiliza un arreglo unidimensional (como un histograma), puede utilizarse una Grid 1D. Si se trabaja con imágenes o matrices (como en la multiplicación de matrices o la aplicación de filtros), puede utilizarse una Grid 2D.

Si configuran bloques de 256 hilos y necesitan procesar un arreglo de 1,000,000 de elementos, ¿cuántos bloques se requieren para cubrir todos los datos?

Bloques = ⌈1 000 000 / 256⌉ = ⌈3906.25⌉ = 3907 bloques. Con 3906 bloques solo se cubren 999 936 elementos, por lo que se necesita un bloque más para los 64 restantes. Se lanzan 1 000 192 hilos; los 192 sobrantes no hacen nada gracias a la condición if i < n and j < n.

¿Qué consideraciones tomarán en cuenta para definir el tamaño y la configuración de la Grid de su algoritmo?

Como el resultado es una matriz C de 1000 × 1000, se usa una Grid 2D en la que cada hilo corresponde a una posición (i, j) de C. Los bloques son de 16 × 16 = 256 hilos: es múltiplo de 32 (el tamaño de un warp) y está por debajo del máximo de 1024 hilos por bloque. Para cubrir toda la matriz, la Grid tiene ⌈1000/16⌉ × ⌈1000/16⌉ = 63 × 63 bloques, en total 1 016 064 hilos. Como 1000 no es múltiplo de 16, sobran hilos en los bordes, y la condición if i < n and j < n evita que escriban fuera de la matriz.

Identificación del Kernel: en CUDA, el kernel es una función que se ejecuta en la GPU mediante múltiples hilos en paralelo. A diferencia de la implementación secuencial, donde

generalmente se utilizan ciclos `for` para recorrer y procesar los datos, en CUDA cada hilo ejecuta el kernel y trabaja exclusivamente sobre los datos que le corresponden, utilizando su identificador global.

En esta etapa, deberán identificar las operaciones del algoritmo secuencial que pueden ejecutarse en paralelo y transformarlas en una función CUDA (kernel).

¿Qué operación(es)/función(es) del algoritmo seleccionado se ejecutarán dentro del kernel?

Dentro del kernel multiplicar_paralelo se ejecuta el producto punto entre la fila i de A y la columna j de B: el ciclo for k in range(n): suma += A[i, k] * B[k, j]. Los dos ciclos externos de la versión secuencial (for i, for j) se reemplazan por la Grid de hilos: cada hilo obtiene su (i, j) con cuda.grid(2).

¿Qué resultado generará cada hilo al ejecutar el kernel?

Cada hilo calcula un solo elemento de la matriz resultado, C[i, j] = Σ□ A[i, k] · B[k, j], y lo escribe en la memoria global de la GPU. En total, 1 000 000 de hilos generan los 1 000 000 de elementos de C.

Transferencia de resultados de la GPU al Host: después de ejecutar el kernel, los resultados permanecen almacenados en la memoria de la GPU (VRAM). Para que el programa en Python pueda utilizarlos, mostrarlos o guardarlos, es necesario transferirlos nuevamente a la memoria RAM del Host.

¿Qué instrucción o método de CUDA Python permite copiar los resultados desde la GPU hacia el Host?

El método copy_to_host() del arreglo en el dispositivo: C_gpu = dC.copy_to_host(). Copia la matriz C desde la VRAM a un arreglo de NumPy en la RAM del Host.

Debido a que la ejecución de los kernels en CUDA es asíncrona, es importante utilizar cuda.synchronize() para asegurar que las operaciones pendientes en la GPU hayan finalizado antes de medir su tiempo de ejecución.

Para medir únicamente el tiempo de transferencia de regreso al Host, deberán iniciar el cronómetro después de que el kernel haya finalizado y detenerlo una vez completada la copia de los resultados.

¿Cuánto tiempo tomó transferir los resultados desde la GPU hacia el Host?

Transferir la matriz C (4 000 000 bytes ≈ 3.81 MB) desde la GPU hacia el Host tomó aproximadamente 1.5 ms (promedio de 3 ejecuciones: 1.395, 1.521 y 1.580 ms). El cronómetro se inició después de cuda.synchronize() para medir solo la copia y no el kernel.

VI. Discusión de Resultados y Conclusiones

Para determinar si la implementación paralela con CUDA ofrece una mejora de rendimiento respecto a la versión secuencial en Python, calculen la aceleración (Speedup) dividiendo el tiempo de ejecución de la CPU entre el tiempo total de ejecución de CUDA.

El tiempo total de CUDA debe incluir la transferencia de datos hacia la GPU, la ejecución del kernel y la transferencia de los resultados de regreso al Host.

¿Cuál fue la aceleración obtenida? Muestren sus cálculos e indiquen si la implementación paralela mejoró el rendimiento respecto a la versión secuencial.

Con n = 1000:

Tiempo secuencial (CPU, Python): 186.542 s
Tiempo total de CUDA (promedio de 3 ejecuciones) = transferencia Host→GPU (1.734 ms) + kernel (9.608 ms) + transferencia GPU→Host (1.499 ms) = 12.841 ms = 0.012841 s
Speedup = 186.542 s / 0.012841 s ≈ 14 527×

La implementación paralela sí mejoró el rendimiento: fue unas 14 500 veces más rápida que la versión secuencial. Las transferencias (≈ 3.2 ms) representan cerca del 25 % del tiempo total de CUDA, pero aun así no eliminan la ventaja, porque el cálculo (10^9 operaciones) es mucho más pesado que los datos que se mueven (≈ 11.4 MB).

¿Cuál fue la aceleración obtenida? Muestren sus cálculos e indiquen si la implementación paralela mejoró el rendimiento respecto a la versión secuencial.

La principal dificultad fue medir los tiempos de la GPU correctamente. Al principio la primera ejecución salía mucho más lenta, porque la primera llamada al kernel incluye la compilación JIT de Numba y la primera transferencia inicializa el contexto de CUDA. Además, como los kernels son asíncronos, el cronómetro se detenía antes de que la GPU terminara. Lo resolvimos con una ejecución de calentamiento antes de medir y con cuda.synchronize() antes de detener el cronómetro. Otra dificultad fue pasar de los tres ciclos anidados a una Grid 2D: había que asignar a cada hilo su posición (i, j) con cuda.grid(2) y proteger los bordes con if i < n and j < n, porque 1000 no es múltiplo de 16.

¿Cuál fue la principal dificultad que encontraron al transformar el algoritmo secuencial en una implementación paralela utilizando CUDA? Expliquen cómo lograron resolverla.

La principal dificultad fue configurar el entorno: conda no estaba instalado y, con solo numba instalado por pip, el kernel fallaba porque no encontraba las librerías de CUDA. Lo resolvimos creando un entorno virtual con python3 -m venv e instalando numba-cuda[cu12] y cupy-cuda12x, que traen el toolkit de CUDA 12 incluido.